GPU-Accelerated Pairwise Sequence Alignment

Using Pair-HMM Viterbi with Affine Gap Penalties and GACT Tiling

ECE 213 — Final Project Report

Abhinandan Sharma (A69041851) | Sanskriti Goyal (A69041145)

1. Background and Problem Definition

1.1 Pairwise Sequence Alignment

Pairwise sequence alignment finds the optimal correspondence between two biological sequences — DNA, RNA, or protein strings — accounting for matches, mismatches, and gaps. It reveals evolutionary relationships, identifies conserved regions, and supports variant calling, phylogenetic analysis, and genome assembly.

An alignment transforms sequences A and B into equal-length gapped sequences A' and B' by inserting gap characters ('−'), where each column represents a match, mismatch, or gap event. For example:

```
Sequence A:          ACGTTAGC
Sequence B:          ACGAGCA
Aligned Sequence A: ACGTTAGC-     [reference]
Aligned Sequence B: ACG--AGCA     [Query]
Edits:               6 matches, 2 deletions and 1 insertion in B
```

Alignment quality is measured by a scoring function. Biologically realistic models use **affine gap penalties** — opening a gap costs more than extending one — reflecting the tendency for evolution to produce longer contiguous indels rather than many scattered single-base events.

1.2 Pair Hidden Markov Models (Pair-HMM)

A Pair-HMM models the joint generation of two sequences through three hidden states:

- **M (Match/Mismatch):** both sequences emit one character — diagonal move
- **I (Insert):** only the query emits — left move (gap in reference)
- **D (Delete):** only the reference emits — upward move (gap in query)

Key model parameters:

Parameter	Value	Meaning
Emission: p(match | M)	0.90	Probability two aligned bases are identical
Emission: p(mismatch | M)	0.10	Probability two aligned bases differ
Transition: M → M	0.90	Stay in match state
Transition: M → I	0.05	Open insertion from match
Transition: M → D	0.05	Open deletion from match

Transition: I → M	0.70	Close insertion, return to match
Transition: I → I	0.30	Extend insertion
Transition: D → M	0.70	Close deletion, return to match
Transition: D → D	0.30	Extend deletion
I → D / D → I	Forbidden	Avoids alignment ambiguity
Initial: π(M)	0.90	Start in match state
Initial: π(I)	0.05	Start in insert state
Initial: π(D)	0.05	Start in delete state

Opening a gap costs log(0.05/0.90) ≈ −2.89 nats; extending costs log(0.30/0.70) ≈ −0.85 nats — implicitly encoding affine gap penalties. All probabilities are computed in log-space to prevent numerical underflow over long sequences:

```
log P(x) + log P(y)  replaces  P(x) * P(y)
max(log P1, log P2)  replaces  max(P1, P2)  [for Viterbi]
```

1.4 The Viterbi Algorithm

Viterbi finds the most probable alignment path via dynamic programming over three tables V_M, V_I, V_D — one per state — where each cell holds the log-probability of the best partial alignment ending at position (i, j):

```
V_M[i][j] = log_emit(r[i], q[j]) + max_s( V_s[i-1][j-1] + logT[s→M] )
V_I[i][j] =                        max( V_M[i][j-1] + logT[M→I],
                                        V_I[i][j-1] + logT[I→I] )
V_D[i][j] =                        max( V_M[i-1][j] + logT[M→D],
                                        V_D[i-1][j] + logT[D→D] )
```

Traceback must record both the direction (DIAG/UP/LEFT) and the predecessor state at each cell — six values per cell versus one in simple Needleman-Wunsch.

1.5 The GACT Tiling Heuristic

For long sequences, filling the full O(mn) DP matrix is too costly. GACT divides the alignment into T×T tiles along the diagonal. At each tile boundary, it scans an O×O overlap corner and picks the highest-scoring cell as the seed for the next tile — adaptively tracking the alignment path even when it drifts from the main diagonal.

1.6 Motivation for GPU Acceleration

Full DP over 5,000 sequence pairs (up to 10,000 bp each) takes ~1 hour on a single CPU core. The O(mn) inner loop is inherently data-parallel along antidiagonals, making GPUs a natural fit. Two datasets were used: a general **sequences dataset (5,000 pairs, 200–10,000 bp)** and a **SARS-CoV-2 genome collection (1,000 pairs from GISAID, including IUPAC ambiguity characters)**. The goal was to implement and validate GPU Viterbi alignment against both datasets, then optimise for throughput while keeping score loss within an acceptable threshold relative to the full-DP reference.

2. Prior Work

GPU acceleration of pairwise alignment has been extensively studied. CUDASW++ (Liu et al., 2009–2013) demonstrated Smith-Waterman speedups using inter-query and intra-query anti-diagonal parallelism. GASAL2 (Ahmed et al., 2019) extended this to global, local, and semi-global modes with traceback, achieving up to 21x speedup over Parasail.

Pair-HMM acceleration has been driven largely by GATK HaplotypeCaller. Intel GKL provided AVX-512 CPU implementations, while Ren et al. (2018) and Li et al. (2021) demonstrated GPU speedups for the forward algorithm. Schmidt et al. (2024) achieved 2.62 TCUPS on an NVIDIA L40S using wavefront parallelism and warp-shuffle reductions, outperforming prior GPU work by 8.6x and Intel GKL by over 50x. Crucially, all prior Pair-HMM GPU work targets the forward algorithm — Viterbi alignment with traceback remains unexplored.

ADEPT (Awan et al., 2020) inspired the GACT tiling approach used here. AGAThA (Park et al., 2024, PPoPP) addressed load imbalance via dynamic workload balancing; we adopt length-based sorting as a practical mitigation.

3. Parallelisation Approach

We implemented three successive levels of GPU parallelism, each building incrementally on the previous:

3.1 Level 0 — Single GPU Thread (Baseline)

The first GPU implementation used a single CUDA thread (blockIdx.x=0, threadIdx.x=0) executing all pairs serially in a for-loop. This served as the correctness baseline: the algorithm was identical to the CPU implementation, but all data resided in GPU global memory. The serial inner loop over diagonal cells meant no parallelism was exploited.

3.2 Level 1 — One Block Per Pair (Inter-Pair Parallelism)

The outer pair for-loop was eliminated. Instead, each sequence pair was assigned its own CUDA thread block: pair = blockIdx.x. All blocks execute simultaneously on the GPU's SMs, making all pairs run in parallel.

Key implementation details:

- numPairs CUDA blocks launched, blockSize = 1 thread per block
- The per-pair traceback scratch buffers (d_tbDir, d_tbState) were scaled to numPairs * 3 * T * T, with each block accessing its own private region via pointer offset: tbDir = d_tbDir + pair * 3 * CELL
- Tile-loop scalars (carryLogProb, reference_idx, etc.) became thread-local register variables since only one thread per block uses them

Runtime result: ~43,716 ms — approximately 10x speedup over the CPU single-thread version. The A30 GPU has 56 SMs each capable of running 32 blocks simultaneously, giving up to 1,792 concurrent pairs. The speedup is lower than the theoretical maximum due to load imbalance: sequences range from 200 to 10,000 bp, so blocks handling long sequences take 50x longer than short ones, leaving many SMs idle.

3.3 Level 2 — Wavefront Parallelism (Intra-Pair Parallelism)

Within each block, the inner cell loop over each anti-diagonal was parallelised across threadIdx.x threads. The block size was increased to 256 threads. Each thread handles one or more cells on the current diagonal using the stride pattern identical to the reference NW parallel implementation:

```
for (int i = i_start + tx; i <= i_end; i += blockDim.x) {
    int j = k - i;
    // compute vm, vi, vd for this cell
}
__syncthreads();   // barrier before next diagonal
```

The barrier after each diagonal is mandatory: diagonal k+1 reads wavefront values written by diagonal k, so all threads must complete their writes before any thread reads for the next diagonal.

Additional changes required by this parallelism:

- Wavefront arrays (wf_M, wf_I, wf_D) moved to shared memory and initialised in parallel using a strided loop
- Sequence segments loaded into shared memory (shared_ref, shared_qry) cooperatively by all threads once per tile
- Overlap tracking: each thread accumulates its own local best overlap score in registers throughout the entire wavefront loop; a parallel tree reduction after the loop finds the global best
- Traceback remains serial (tx==0 only) since each step depends on the previous
- Coalesced write of the reversed traceback path to global memory by all threads: for (k = tx; k < localLen; k += blockDim.x)

Runtime result: ~10,844 ms — approximately 4x speedup over Level 1. Note that the parallel tree reduction uses a tie-breaking rule matching the serial CPU/single-thread behaviour: on equal scores, the higher state index wins (D > I > M), preserving the >= comparison with s iterating 0,1,2.

3.4 Level 3 — Removing Redundant Global Memory Writes

After careful analysis, we observed that the tbDir (direction) array for inner cells is always constant given the HMM state: state M always produces DIR_DIAG, state I always produces DIR_LEFT, state D always produces DIR_UP. This is a structural property of the HMM, not data-dependent. Therefore, tbDir writes for inner cells are entirely redundant.

We eliminated all tbDir writes for inner cells and modified the traceback to derive direction from cur_state directly:

```
// Instead of reading tbDir[cur_state * CELL + cell]:
if      (cur_state == S_M) { dir = DIR_DIAG; ti--; tj--; }
else if (cur_state == S_I) { dir = DIR_LEFT; tj--;        }
else                       { dir = DIR_UP;   ti--;        }
prev_state = tbState[cur_state * CELL + cell]; // only this read needed
```

This reduced the global memory write count per inner cell from 6 to 3 (3 tbDir writes eliminated), and replaced each traceback's random global tbDir read with a free register comparison.

Runtime result: ~4,200 ms — approximately 2.6x further speedup over Level 2. Since the wavefront loop is the hottest code path (400 diagonals x ~200 cells x ~100 tiles), halving its global write traffic has a proportional impact.

4. Validating Correctness

4.1 Full DP Matrix Reference Implementation

Since no external ground-truth reference existed for pair-HMM Viterbi alignments with our specific parameters, we first implemented a CPU full-DP version (alignment_pairhmm_full_DP.cpp) that solves the complete O(mn) Viterbi DP matrix for each pair without any tiling heuristic. This produces provably optimal alignments under the pair-HMM model and served as our gold standard.

Full-DP runtime: approximately 1 hour for 5,000 pairs on the **sequences.fa dataset**, and approximately 2 hours for 1,000 pairs on the **SARS-CoV-2 dataset**. This runtime was acceptable for a one-time reference generation but entirely impractical for production use.

4.2 GACT CPU Validation

We then implemented the CPU GACT version (alignment_CPU_execution.cpp) using the same pair-HMM model but with T=200, O=64 tiling. Comparing its output to the full-DP reference using the provided compare_alignment tool (which scores both alignments and reports percentage score loss):

Dataset	Pairs	CPU Full-DP Time	GACT CPU Time	Score Loss
sequences.fa	5,000	~60 min	~4,000 ms	0.056%
SARS-CoV-2 (20k)	1,000	~120 min	~800 ms	3.6%

The 0.056% loss on the main dataset confirmed that GACT with T=200, O=64 is an excellent heuristic. The higher 3.2% loss on the SARS-CoV-2 dataset required further investigation.

4.3 SARS-CoV-2 Dataset — Non-Standard Characters

SARS-CoV-2 sequences from GISAID contain ambiguity codes beyond the standard ACGT alphabet. For example:

```
>Switzerland/SO-ETHZ-500145/2020|OU000199.2|2020-11-12
NNNN
```

The 3.2% loss on SARS-CoV-2 sequences is therefore primarily attributable to ambiguity characters, not a defect in the alignment algorithm. A production system would pre-process sequences to replace ambiguity codes with the most likely base or mask them.

4.4 GPU Correctness Validation

For every GPU implementation (Level 0 through Level 3), we verified correctness using both provided tools:

- check_alignment: confirms that the aligned sequences, when gaps are removed, exactly reproduce the original unaligned sequences. Zero failures in all implementations.
- compare_alignment: compares alignment quality scores against the GACT CPU reference. All GPU implementations produced 0.056% loss on the sequences.fa dataset, matching the CPU GACT baseline exactly.

One subtle correctness issue encountered during development was the tie-breaking rule in the parallel reduction. The serial CPU implementation iterates states in order s=0,1,2 with >= comparison, so on equal scores the last state (S_D=2) wins. The parallel reduction must replicate this exactly to preserve bit-identical results:

```
bool take = (s_bestScore[tx+stride] > s_bestScore[tx]) ||
            (s_bestScore[tx+stride] == s_bestScore[tx] &&
             s_best_st[tx+stride] >= s_best_st[tx]);
```

5. Methodology and Results

5.1 Hardware and Software Environment

All GPU experiments were conducted on the UCSD DSMLP cluster (dsmlp-login.ucsd.edu) using:

Component	Specification
GPU	NVIDIA A30 (Ampere, 56 SMs, 224 CUDA cores/SM, 24 GB HBM2)
CUDA	11.x with nvcc
CPU	8 cores allocated per job
Container	yatisht/ece213-wi26:latest
Build system	CMake with -O3 optimisation
Test dataset	sequences.fa — 5,000 pairs, lengths 200–10,000 bp
Batch size	500 pairs per launch

5.2 Runtime Progression

Optimization	Description	sequences Dataset (assignment2) [5000 Pairs], [batchSize=625]			Sars Cov2 Dataset [1000 Pairs], [batchSize=125]		
		Runtime (ms)	Speedup	Loss %	Runtime (ms)	Speedup	Loss %
CPU Full-DP	Full O(mn) DP, no tiling	3,600,527	1x (baseline)	0.000%	6,589,550	1x (baseline)	0.0%
CPU GACT	Tiled GACT, **T=200, O=64**	90,749	40x vs CPU full DP	0.056%	54,697	~120x vs CPU full DP	3.6%
GPU 1-Thread	Single GPU thread, all pairs serial	4,035,560	0.02x vs CPU GACT	0.056%	2,452,100	~0.02x vs CPU GACT	3.6%
GPU Level 1	One block per pair, 1 thread/block	43,716	~2x vs CPU GACT	0.056%	24,447	~2.4x vs CPU GACT	3.6%
GPU Level 2	Level 1 + wavefront parallelism (256 threads)	10,844	~4x vs Level 1	0.056%	4,954	~5x vs Level 1	3.6%
GPU Level 3	Level 2 + remove redundant tbDir writes + batchSize optimization	4,200	~2.6x vs Level 2	0.056%	2,977	1.6x vs Level 2	3.6%
Final Speedup wrt CPU GACT			**21.6x**			**18.37x**	

5.3 Why the GPU 1-Thread Version Is Slower than CPU

The single-thread GPU version is ~50x slower than the CPU GACT despite identical algorithms. The reasons are:

- GPU memory latency: every global memory access from a single thread experiences full DRAM latency (~300-600 ns) with no latency hiding from other warps.
- Low SM utilisation: one thread uses 1/2048 of one SM's capacity, leaving 99.95% of compute resources idle.
- PCIe transfer overhead: sequences and results must be transferred between host and device memory before and after the kernel, adding latency that is not amortised without parallelism.

This implementation exists purely as a correctness stepping-stone — it proved the HMM logic was correct on GPU before parallelism was introduced.

5.4 Attempted Optimisation: float vs. double

We experimentally tested replacing all double precision values with single precision float. The A30 GPU provides 32x higher float32 throughput than float64. However, this change resulted in a slight runtime increase (+100ms) and a marginally higher loss percentage. Analysis showed the kernel is dominated by the __syncthreads() overhead (one per diagonal, ~400 per tile, ~100 tiles per long pair = ~40,000 syncs per pair), not by FP arithmetic throughput. Reducing precision did not reduce the sync count and introduced slight score perturbations from floating-point rounding. The float optimisation was therefore not adopted.

6. Use of AI Declaration

Artificial intelligence tools were used in the following limited capacities during this project:

1. HMM parameter generation: The specific numerical values of the HMM transition and emission log-probabilities (e.g., LOG_MM = -0.10536051565782630 = log(0.90)) were verified using an AI assistant to ensure floating-point accuracy of the constants.
2. Code readability: AI assistance was used to add inline comments and section headers to the final GPU kernel code to improve readability for the report submission.All DP recurrences, traceback implementations, CUDA parallelisation patterns, and correctness fixes were written by the project team.
3. Report writing assistance and literature review: This report was drafted with AI assistance for structuring and prose clarity, while all technical content, experimental results, and analysis reflect the team's own work.

All core algorithmic contributions — the pair-HMM Viterbi CPU implementation, the GACT tiling, the GPU parallelisation at all three levels, the traceback correctness fixes, and the parallel reduction tie-breaking fix — were independently developed and validated by the project team.

7. References

[1] Needleman, S.B., Wunsch, C.D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. Journal of Molecular Biology, 48(3), 443-453.

[2] Durbin, R., Eddy, S., Krogh, A., Mitchison, G. (1998). Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids. Cambridge University Press.

[3] Daily, J. (2016). Parasail: SIMD C library for global, semi-global, and local pairwise sequence alignments. BMC Bioinformatics, 17(1), 81.

[4] Awan, M.G., Deslippe, J., Buluc, A., et al. (2020). ADEPT: A domain independent sequence alignment strategy for GPU architectures. BMC Bioinformatics, 21(1), 1-29.

[5] Ahmed, N., Lévy, J., Ren, S., et al. (2019). GASAL2: a GPU accelerated sequence alignment library for high-throughput NGS data. BMC Bioinformatics, 20(1), 520.

[6] Ren, S., Bertels, K., Al-Ars, Z. (2018). Efficient Acceleration of the Pair-HMMs Forward Algorithm for GATK HaplotypeCaller on Graphics Processing Units. Evolutionary Bioinformatics, 14.

[7] Banerjee, S.S., et al. (2017). On accelerating pair-HMM computations in programmable hardware. 27th International Conference on Field Programmable Logic and Applications (FPL). IEEE.

[8] Li, E., Banerjee, S.S., Huang, S., Iyer, R.K., Chen, D. (2021). Improved GPU implementations of the Pair-HMM forward algorithm for DNA sequence alignment. IEEE 39th ICCD, 299-306.

[9] Zhao, H., et al. (2019). GPU accelerated sequence alignment with traceback for GATK HaplotypeCaller. BMC Genomics, 20(Suppl 2), 184.

[10] Ligowski, L., Rudnicki, W. (2009). A tile-based parallel Viterbi algorithm for biological sequence alignment on GPU with CUDA. IEEE IPDPS, 1-8.

[11] Schmidt, A., et al. (2024). gpuPairHMM: High-speed Pair-HMM Forward Algorithm for DNA Variant Calling on GPUs. arXiv:2411.11547.

[12] Park, S., et al. (2024). AGAThA: Fast and Efficient GPU Acceleration of Guided Sequence Alignment for Long Read Mapping. ACM PPoPP 2024.

[13] Zhong, H., Pan, X., He, Z., et al. (2025). GPU acceleration for DNA sequence alignment algorithm and its application. CCF Transactions on High Performance Computing.

[14] Liu, Y., Schmidt, B., Maskell, D.L. (2009). CUDASW++: Optimizing Smith-Waterman sequence database searches for CUDA-enabled graphics processing units. BMC Research Notes, 2(1), 73.

[15] NVIDIA Corporation. (2023). CUDA C++ Best Practices Guide. NVIDIA Developer Documentation.